

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Estructura de Datos



Manual Técnico Practica 01

Estudiante: Juan Fernando García Natareno

RA: 202230077

El presente documento constituye el Manual Técnico del proyecto Práctica #1 del Laboratorio de Estructura de Datos, correspondiente a la carrera de Ingeniería en Ciencias y Sistemas del Centro Universitario de Occidente (CUNOC) de la Universidad San Carlos de Guatemala.

El proyecto consiste en el desarrollo de una versión avanzada del juego de cartas UNO, implementado en el lenguaje C++ bajo el paradigma de Programación Orientada a Objetos. El sistema fue diseñado para ejecutarse en consola de texto, ofreciendo una experiencia de juego completa con soporte para múltiples jugadores, reglas configurables y cartas especiales personalizadas.

Para la construcción del sistema se implementaron estructuras de datos desde cero, sin el uso de librerías estándar de contenedores. Las estructuras utilizadas incluyen una Pila para el manejo del mazo principal y la mesa de jugadas, una Lista Enlazada Simple para la mano de cada jugador, y una Lista Enlazada Circular Doble para la gestión de turnos entre jugadores.

La arquitectura del sistema se divide en capas bien definidas: la capa de datos conformada por las clases de cartas y nodos, la capa de estructuras que implementa los TADs, la capa de lógica que maneja las reglas del juego, y la capa de control representada por el motor del juego que coordina todos los componentes.

El sistema permite configurar antes de iniciar cada partida diversas reglas opcionales como la acumulación de cartas, el reto del +4, el modo de robo y la posibilidad de ganar con carta negra. Adicionalmente se implementaron dos cartas personalizadas que enriquecen la experiencia de juego más allá de las reglas estándar del UNO.

TAD Pila – Mazo Principal

Este TAD gestiona las cartas que han sido jugadas durante la partida, permitiendo reciclarlas al mazo cuando este se vacía.

Atributos privados:

- NodoCarta* cabeza: Puntero al tope de la pila, representa la carta disponible para robar
- Int size: Contador que registra el número total de cartas en el mazo

Métodos públicos:

- insertarCarta(Carta*): Recibe un puntero a una carta y la cola en el tope de la pila
- eliminarCarta(): Extrae y devuelve la carta del tope, se usa cuando un jugador roba
- mostrarData(): Retorna la carta del tope sin eliminarla.
- estaVacia(): Devuelve un booleano indicando si ya no hay cartas.
- getSize(): Retorna el número total de cartas en el mazo.
- llenarMazo(int): Crea tantas cartas según la cantidad de jugadores y las baraja antes de apilarlas

TAD Lista Enlazada Simple – ManoJugador

Este TAD representa la mano de cada jugador, permitiendo agregar, eliminar, ordenar las cartas de forma dinámica.

Atributos privados:

- NodoMano* cabeza: Puntero al primer nodo de la lista, representa la primera carta de la mano.
- Int size: Contador que registra el número de cartas en la mano del jugador

Métodos públicos

- compararCarta(Carta*, Carta*): Comparamos dos cartas por color y número para el ordenamiento. Retorna entero negativo, cero o positivo
- agregarCarta(Carta*): Insertar una carta al inicio de la lista y reordena automáticamente con Shell Sort.
- eliminarCarta(int): Recorre la lista hasta la posición indicada y elimina el nodo retornando la carta.
- mostrarCartas(): Recorre e imprime todas las cartas numeradas.
- ordenarCartas(): Implementa Shell Sort sobre un arreglo temporal, ordenando primero por color y luego por número
- estaVacio(): Verifica si el jugador no tiene cartas.
- getSize(): Retorna el número de cartas en la mano.

TAD Lista Enlazada Circular Doble – ListaJugadores

Dado que los turnos son rotativos, se utiliza una Lista Enlazada Circular Doble que permite avanzar en ambas direcciones sin perder el hilo del juego.

Atributos privados:

- NodoJugador* cabeza: Puntero al primer jugador registrado en la lista.
- NodoJugador* jugadorActual: Puntero al jugador cuyo turno está activo actualmente.
- int size: Contador que registra el número total de jugadores.
- bool sentidoHorario: Booleano que determina la dirección del juego, true para horario y false para antihorario.

Métodos públicos:

- agregarJugador(string): Inserta un nuevo jugador al final del ciclo circular.
- avanzarTurno(): Mueve jugadorActual al siguiente o anterior nodo según sentidoHorario.
- saltarTurno(): Llama a avanzarTurno dos veces para omitir al siguiente jugador, usado por la carta Salto.
- cambiarSentido(): Invierte el booleano sentidoHorario, usado por la carta Reversa.
- getJugadorActual(): Retorna el jugador cuyo turno es el actual usando jugadorActual.
- getCabeza(): Retorna el puntero cabeza para recorrer la lista.
- estaVacía(): Verifica si no hay jugadores registrados.
- getSize(): Retorna el número total de jugadores.



